

Containers and Kubernetes

Before class: Install Docker and K3s

Complete this setup before the two-hour class. Allow roughly 20–30 minutes, depending on download speed.

Use the Ubuntu 24.04 VirtualBox VM you installed with the TAs in the previous tutorial. The TAs will not provide a preconfigured VM.

1. Check the existing Ubuntu VM

1. Sign in, open Terminal, confirm `sudo` works, and check internet access inside Ubuntu.
2. Keep the existing VirtualBox NAT network.
3. The VM needs at least **2 virtual CPUs**, **4 GiB RAM**, and **6 GiB free space** under `/var` (a 20 GiB virtual disk is recommended).
4. **Note:** Shut down Ubuntu before changing CPU or RAM settings in VirtualBox Settings.

2. Download and run the setup script

1. Download `preclass-setup.sh` using the browser inside Ubuntu and save it in `Downloads`.
2. Open Terminal with `Ctrl + Alt + T`, then run:

```
cd ~/Downloads
ls -l preclass-setup.sh
sudo bash preclass-setup.sh --install
```

Note: Enter your Ubuntu password at the `sudo` prompt; no characters will appear on screen while you type. Keep the VM connected to the internet until setup finishes. If you saved the script elsewhere, change to that folder first.

3. Understand the boundary of the script

The script performs the following setup tasks:

- Installs **Docker Engine**, the **Docker Compose plugin**, and a single-node **K3s cluster**.
- Caches the public `nginx` and `BusyBox` images and checks both runtimes.

It **does not** create a course working directory, application files, Compose file, Kubernetes manifest, shell helper, or `cloud-demo` image. You will create every course file from this handout during class.

- **Expected observation:** The final output includes a `Ready K3s node`, `READY for class`, and `Setup complete`.
- If installation stops, keep the error text and contact a TA before class.

Installation references:

- [Docker Engine Installation for Ubuntu](#)
- [K3s Quick-Start Guide](#)

4. Verify the installation before class

From the folder containing the script, run the read-only readiness check. With no options, the script defaults to `--check`. It does not create course workloads or files.

```
sudo bash preclass-setup.sh --check
sudo docker version
sudo docker compose version
sudo k3s kubectl get nodes
```

- **Expected observation:** Docker lists Client and Server versions, Compose prints a version, the K3s node status is `Ready`, and the script prints `READY for class`. Keep the script so you can rerun `--check` after restarting the VM.

Troubleshooting: If setup does not finish

Symptom	Next Step
File not found	Use Ubuntu Files to locate the download, open Terminal in that folder, and rerun the command.
Package manager is locked	Let Software Updater or another installation finish, then rerun <code>--install</code> . Do not delete lock files.
Download or DNS error	Check internet access inside Ubuntu and the VM NAT adapter, then rerun <code>--install</code> .
Firewall, package conflict, or existing K3s	Keep the error and ask a TA to review the VM. Do not remove packages at random.
Node not Ready / service failed	Wait one minute after boot and rerun <code>--check</code> . If it still fails, show the status commands below to a TA.

```
sudo systemctl status k3s --no-pager
sudo journalctl -u k3s -n 50 --no-pager
```

At the Start of the Lab

Open two Ubuntu terminals on the same VM. In **BOTH** terminals, create and enter your own working directory, then type the `k` function exactly once. It is a convenience shortcut for the longer `K3s` command in that terminal session; no `lab-shell.sh` file is supplied or sourced.

```
mkdir -p ~/container-tutorial-lab
cd ~/container-tutorial-lab
k() { sudo k3s kubectl -n container-lab "$@"; }
sudo docker version
sudo k3s kubectl get nodes
```

Terminal Setup:

- **Terminal A** will later run a `watch` or `port-forward` command.
- Use **Terminal B** for standard interactive commands.
- Stop any foreground command using `Ctrl + C`.
- Every `localhost` or `127.0.0.1` address refers to Ubuntu, so use the web browser inside the VM.
- **Important:** If you open a new terminal at any point, repeat the `cd` command and redefine the `k` function.

Lab A (Part 1): Create the v1 Application Files

In **Terminal B**, stay in `~/container-tutorial-lab`. Our application is a static website served by `nginx`. Copy each command block in its entirety, including the final `EOF` line. You may personalize the `<h1>` heading, but keep `Version: v1` intact so later automated checks work.

```
mkdir -p app/v1
cat > app/v1/index.html <<'EOF'
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<title>My Cloud App</title>
<h1>Hello from my application!</h1>
<p>Version: v1</p>
</html>
EOF
```

`index.html` is the content returned to a browser. `nginx.conf` configures `nginx` to listen on port 80, serve the page, and attach the container's hostname as a response header:

```
cat > app/v1/nginx.conf <<'EOF'
server {
    listen 80;
    server_name _;
    add_header X-Pod-Name $hostname always;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
EOF
```

Dockerfile is the image recipe. FROM uses the public nginx base image cached before class. COPY adds the two files you created:

```
cat > app/v1/Dockerfile <<'EOF'
FROM nginx:stable-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html /usr/share/nginx/html/index.html
EOF

ls -l app/v1
```

- **Expected observation:** Dockerfile, index.html, and nginx.conf exist in app/v1. These are source files; cloud-demo:v1 does not exist until you build it in Part 2.
- **Syntax Note:** The quoted <<'EOF' writes the included text literally (preserving variables like \$hostname). EOF must sit alone on its own line. The > operator overwrites the file if you rerun the command.

Lab A (Part 2): Build, Run, and Inspect v1

Build before running

The path app/v1 is the **build context**: the set of files Docker can read for this build. --pull=false uses the cached base image, and -t assigns the image name and tag.

```
sudo docker image inspect nginx:stable-alpine --format '{{.Id}}'
sudo docker build --pull=false -t cloud-demo:v1 app/v1
sudo docker image inspect cloud-demo:v1 --format '{{.Id}}'
```

- **Expected observation:** Both `inspect` commands print `sha256` image IDs in the terminal, and the build succeeds. If `nginx:stable-alpine` is missing, rerun the pre-class `--install` script while online. Fix any Dockerfile or path errors before continuing.

Run your image

`--pull=never` makes a missing local build explicit. `-d` runs the container in the background; `--name` assigns a CLI identifier; `-p 127.0.0.1:8081:80` maps Ubuntu host port 8081 to container port 80.

```
sudo docker run --pull=never -d --name cloud-web \
  -p 127.0.0.1:8081:80 cloud-demo:v1
curl -i http://127.0.0.1:8081
sudo docker ps
```

- **Expected observation:** The HTTP response contains your heading, `Version: v1`, and an `X-Pod-Name` header. You can also open `http://127.0.0.1:8081` in the Ubuntu web browser.

Inspect the container

```
sudo docker logs --tail 10 cloud-web
sudo docker exec cloud-web hostname
sudo docker exec cloud-web ps
sudo docker inspect cloud-web
```

Discussion Questions:

1. Which command created files, which built an image, and which started a container?
2. Which operating system kernel executes `nginx` inside the container?

Note: Editing `index.html` does not update an already-built image, and rebuilding an image does not alter a running container. To use a rebuilt image, stop and remove the existing container before starting a new one.

Lab B: Storage, Compose, and Checkpoint

Bind Mount Demonstration

A **bind mount** maps an explicit Ubuntu file path into a container. The temporary BusyBox container below writes into your host directory via `/data` and cleans itself up automatically upon exit (`--rm`).

```
mkdir -p data
sudo docker run --rm \
  --mount type=bind,src="$PWD/data",dst=/data \
  docker.io/library/busybox:1.37.0 \
  sh -c 'echo hello > /data/hello.txt'
cat data/hello.txt
```

- **Expected observation:** `hello` remains in `~/container-tutorial-lab/data` on the host machine after the temporary container exits.

Create and use a Compose file

Docker Compose reads a YAML configuration file that defines service groups. Create the file in Vim (note: YAML indentation using spaces is critical):

```
vim compose.yaml
```

Press `i` to enter Insert mode, then paste the following configuration:

```
services:
  web:
    image: cloud-demo:v1
    pull_policy: never
    ports:
      - "127.0.0.1:8083:80"
```

Press `Esc`, type `:wq`, and press `Enter` to save and exit. (To exit without saving, press `Esc`, type `:q!`, and press `Enter`).

Validate the configuration file before launching the service:

```
sudo docker compose config
sudo docker compose up -d && sleep 1
curl -s http://127.0.0.1:8083 && sleep 1
sudo docker compose down
```

Checkpoint and Pause

Discussion Questions:

- Explain the difference between an **image** and a **container**.
- What does `8081:80` signify?

- Where does `data/hello.txt` physically reside?

Clean up the standalone container before moving to Kubernetes:

```
sudo docker rm -f cloud-web
sudo docker ps -a
```

Note: Explicitly deleted Docker containers stay removed. Kubernetes controllers, on the other hand, actively maintain a requested replica count.

Lab C: Kubernetes Setup & Deployment

Import your v1 image into K3s

Docker and K3s maintain separate image stores. Export the `v1` image built in Docker and import it into K3s:

```
cd ~/container-tutorial-lab
sudo docker save -o /tmp/cloud-demo-v1.tar cloud-demo:v1
sudo k3s ctr -n k8s.io images import /tmp/cloud-demo-v1.tar
sudo rm /tmp/cloud-demo-v1.tar
sudo k3s ctr -n k8s.io images list -q | grep cloud-demo
```

- **Expected observation:** The final command outputs `docker.io/library/cloud-demo:v1`. Note that `k8s.io` here refers to a containerd namespace, which is separate from Kubernetes namespaces.

Create the Kubernetes namespace

```
sudo k3s kubectl create namespace container-lab --dry-run=client -o yaml |
sudo k3s kubectl apply -f -
sudo k3s kubectl get namespace container-lab
```

Reminder: If the `k` command is not recognized, re-declare the function in your current shell session:

```
k() { sudo k3s kubectl -n container-lab "$@"; }
```

Create the Kubernetes manifest

Create `k8s/hello.yaml`. It defines a two-replica **Deployment** and a **ClusterIP Service**. The label `app: hello` connects the Deployment's Pod template to the Service selector.

```
mkdir -p k8s
cat > k8s/hello.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
  namespace: container-lab
spec:
  replicas: 2
  revisionHistoryLimit: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      containers:
        - name: web
          image: docker.io/library/cloud-demo:v1
          imagePullPolicy: Never
          ports:
            - name: http
              containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: http
            initialDelaySeconds: 1
            periodSeconds: 2
---
apiVersion: v1
kind: Service
metadata:
  name: hello
  namespace: container-lab
spec:
  selector:
    app: hello
  ports:
```

```
- name: http
  port: 80
  targetPort: http
type: ClusterIP
EOF
```

Review and apply the manifest

Inspect the YAML contents to identify `replicas`, `image`, `imagePullPolicy`, `matchLabels`, and `readinessProbe` settings:

```
cat k8s/hello.yaml
k apply -f k8s/hello.yaml
k rollout status deployment/hello --timeout=120s
k get deployment,replicaset,pod,service -o wide
k get pods --show-labels
```

- **Expected observation:** Deployment `hello` reports `2/2 ready`, two Pods are running, and Service `hello` receives an assigned ClusterIP address.
- *Troubleshooting:* If the rollout times out, execute `k get pods`, then inspect the failing Pod via `k describe pod <POD_NAME>` and review the `Events` section.

Discussion Questions:

- Which fields connect the Service to the application Pods?
- Which field instructs K3s to use the local image you imported?

Lab D: Service Access and Self-Healing

Access the Service

Path 1: Port Forwarding (Terminal A)

In **Terminal A**, establish a local proxy connection:

```
# Terminal A
k port-forward service/hello 8082:80
```

In **Terminal B**, send a request:

```
# Terminal B
curl -i http://127.0.0.1:8082
```

- **Expected observation:** The output displays `Version: v1. port-forward` routes traffic to a single Pod directly for debugging; it does not perform load balancing across replicas.
- Press `Ctrl + C` in **Terminal A** to stop the port forward.

Path 2: Cluster Internal DNS (Terminal B)

Launch a temporary client Pod inside the cluster to test DNS resolution:

```
k() { sudo k3s kubectl -n container-lab "$@"; }
k run client --image=docker.io/library/busybox:1.37.0 --image-pull-policy=Never --restart=Never --command -- sleep 3600
k wait --for=condition=Ready pod/client --timeout=90s
k exec client -- wget -qO- http://hello
k get endpointslices -l kubernetes.io/service-name=hello
```

- `http://hello` resolves via internal cluster DNS to the Service within the `container-lab` namespace.
- To view HTTP headers, run `k exec client -- wget -S -O- http://hello`. Responses will be served across available Pod backends.

Delete a Pod and observe self-healing

In **Terminal A**, watch the status of Pods in real time:

```
# Terminal A
k get pods -l app=hello -w
```

In **Terminal B**, identify and delete one of the application Pods:

```
# Terminal B
POD=$(k get pods -l app=hello -o jsonpath='{.items[0].metadata.name}')
printf 'Deleting pod: %s\n' "$POD"
k delete pod "$POD"
k rollout status deployment/hello --timeout=120s
k get pods -l app=hello
```

- **Expected observation:** The ReplicaSet controller detects the missing instance and immediately schedules a replacement Pod with a new name to maintain the desired count of 2.
- Press `Ctrl + C` in **Terminal A** to stop watching.

Discussion Questions:

- Which controller object managed the recreation of the deleted Pod?
- Would a standalone Pod or a single-node VM recover automatically in the same way?

Lab E: Scaling and Rolling Updates

Scale the Deployment

```
k scale deployment/hello --replicas=4
k rollout status deployment/hello --timeout=120s
k get pods -l app=hello

k scale deployment/hello --replicas=2
k rollout status deployment/hello --timeout=120s
```

- **Expected observation:** The application scales up to 4 active Pods before scaling back down to 2.

Create and build Application v2

Create a new image version (v2) containing updated page content:

```
cd ~/container-tutorial-lab
mkdir -p app/v2
cp app/v1/Dockerfile app/v1/nginx.conf app/v2/
sed 's/Version: v1/Version: v2/' app/v1/index.html > app/v2/index.html
cat app/v2/index.html

sudo docker build --pull=false -t cloud-demo:v2 app/v2
sudo docker save -o /tmp/cloud-demo-v2.tar cloud-demo:v2
sudo k3s ctr -n k8s.io images import /tmp/cloud-demo-v2.tar
sudo rm /tmp/cloud-demo-v2.tar
```

- **Expected observation:** app/v2/index.html displays Version: v2 . The built image is successfully saved and imported into K3s.
- *Note:* The existing Deployment continues serving v1 until its Pod template image parameter is explicitly updated.

Roll out v2 and perform a rollback

In **Terminal A**, start watching Pod updates:

```
# Terminal A
```

```
k get pods -l app=hello -w
```

In **Terminal B**, update the image specified in the Deployment:

```
# Terminal B
k set image deployment/hello web=docker.io/library/cloud-demo:v2
k rollout status deployment/hello --timeout=120s
k exec client -- wget -q0- http://hello
k rollout history deployment/hello
```

- **Expected observation:** Rolling update strategy (`maxSurge: 1` , `maxUnavailable: 0`) creates new `v2` Pods before terminating `v1` Pods. The client request returns `Version: v2` .

Undo the Deployment (Rollback)

```
k rollout undo deployment/hello
k rollout status deployment/hello --timeout=120s
sleep 2 && k exec client -- wget -q0- http://hello
```

- **Expected observation:** The active image reverts to `v1` .
- Press `Ctrl + C` in **Terminal A** to terminate the watch command.

Discussion Questions:

- Why does updating replicas behave differently than updating the container image template?
- What does a `rollout undo` revert, and what side effects (e.g., database writes) does it leave unchanged?

Lab F: Diagnosis and Cleanup

Diagnostic Commands

Use these core commands when investigating workloads:

```
k get pods
k describe deployment hello
k logs deployment/hello --tail=20
k exec deployment/hello -- hostname
```

Optional Troubleshooting Exercise (After Class)

Trigger an image pull failure intentionally to inspect error handling:

```
k set image deployment/hello web=docker.io/library/cloud-demo:missing
k get pods
# Replace <ACTUAL_POD_NAME> with the identifier of the failing Pod
k describe pod <ACTUAL_POD_NAME>
k rollout undo deployment/hello
k rollout status deployment/hello --timeout=120s
```

Common Symptoms & Fixes

Symptom	Cause & Action
<code>ErrImageNeverPull</code>	Image is missing from K3s store. Build and import the image first. Verify name/tag syntax.
<code>ImagePullBackOff</code>	Remote registry pull failed. Check internet access, registry paths, and credentials via <code>k describe pod</code> .
<code>Pending</code>	Resource limits or scheduling constraints. Run <code>k describe pod</code> and read <code>Events</code> .
<code>Port already in use</code>	A local process or existing container is bound to the port. Stop competing processes or port-forwards.
<code>Service has no ready endpoints</code>	Selector labels do not match Pod labels, or readiness probes are failing. Check <code>Endpoints</code> .
<code>k: command not found</code>	The function definition expired. Re-run: <code>k() { sudo k3s kubectl -n container-lab "\$@"; }</code>

Cleanup After Tutorial

Stop all active processes in open terminals (Ctrl + C). Delete lab resources created during the exercise:

```
k delete -f k8s/hello.yaml
k delete pod client --ignore-not-found
sudo k3s kubectl delete namespace container-lab
sudo docker compose down
sudo docker rm -f cloud-web 2>/dev/null || true
```

Warning: Do not run `docker system prune` or uninstall K3s. Core system services and cached base images should remain intact.

Command Quick Reference

Action	Docker CLI
Build / Deploy	<code>docker build -t NAME:TAG PATH</code>
List Workloads	<code>docker ps</code>
Inspect Logs	<code>docker logs NAME</code>
Execute Command	<code>docker exec NAME CMD</code>
Scale Workload	Manual instance creation
Update / Rollback	Re-create container with new image

Post-Lab Reflection Questions

1. What components are included inside a container image, and what does a container runtime add at execution?
2. How do Linux **namespaces** and **cgroups** differ in container isolation?
3. Why can an image built with Docker Engine run inside K3s even though K3s uses containerd as its runtime?
4. How do **Deployments**, **ReplicaSets**, **Pods**, and **Services** relate to one another?
5. What data persists when a container is removed? What changes when a Pod is recreated by a controller?
6. Which types of failure can two replicas on a single-node cluster tolerate, and which can they not?

Further Reading

- [Docker Architectural Overview](#)
- [Understanding Docker Build Context](#)
- [K3s Documentation](#)
- [Importing Images in K3s](#)
- [Kubernetes Core Concepts](#)
- [Kubernetes Deployments Guide](#)